VARDHAMAN
COLLEGE OF ENGINEERING

A Course End Project Report on

BloodLink: A Database-Driven Blood Donation and Inventory Platform

A9512 - Database Management System Laboratory

Submitted by

W. Nitish	25881A0530
G. Sai Varshith	25881A0544
D. Yogesh	25881A0563

Course Instructor

Ms. Shaik Shahanaz
Assistant Professor
Department of CSE

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

October 2026

VARDHAMAN COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to **JNTUH,**
Approved by **AICTE,** Accredited by **NAAC** with **A**++ Grade
Kacharam, Shamshabad, Hyderabad- 501 218,
Telangana, India.

26 Years of Vibrant Vardhaman

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled **BloodLink-A Database-Driven Blood Donation and Inventory Platform** is carried out by **W.NITISH and G.SAI VARSHITH and D.YOGESH with** Roll Numbers **25881A0530 and 25881A0544 and 25881A0563** towards **A9512 – Data Base Management System Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** during the Academic year 2026-27.

Course Instructor
Ms.Shaik Shahanaz
Assistant Professor, CSE

Academic Head
Dr. R. Karthikeyan
Professor, CSE

Head of the Department
Prof. Y. Vijayalata
Department of CSE

Contents

Abstract 3
1 Introduction 4
1.1 Problem Statement 4
1.2 Scope 4
2 Literature Review 4
3 Objectives 5
4 Methodology 5
4.1 Block Diagram 5
4.2 ER Diagram 5
4.3 Relational Schema 6
4.4 Normalization (3NF / BCNF) 6
4.5 Database Concepts Used 7
4.6 Tools Used 8
4.7 How the Project is Organised 8
5 Implementation 8
5.1 Tables, Constraints and Sequences 8
5.2 Triggers 10
5.3 Sample Data 10
5.4 Constraint Tests 12
5.5 Views 12
5.6 Indexes 13
5.7 Queries (Joins, Aggregates, Nested Queries) 13
5.8 Functions, Procedures and Cursors 15
5.9 Transaction Control 17
5.10 Concurrency Control (Two Sessions) 17
5.11 Test Calls and Data Dictionary Checks 18
6 Execution and Results 19
6.1 Test 1: Table Creation and Data Load 19
6.2 Test 2: Constraints and Trigger Validation 20
6.3 Test 3: SELECT Queries, Joins and Nested Queries 21
6.4 Test 4: Views 23
6.5 Test 5: Indexes 24
6.6 Test 6: Function, Procedures, Cursors and Triggers 24
6.7 Test 7: Transactions (COMMIT, SAVEPOINT, ROLLBACK) 26
6.8 Test 8: Concurrency Control (Two SQL*Plus Sessions) 27
7 Results and Discussion 28
7.1 Summary of Tests 28
7.2 Discussion 28
7.3 Limitations and Future Work 28
8 Mapping POs and SDGs 29
8.1 Program Outcomes (POs) Mapping 29

8.2 Sustainable Development Goals (SDGs) Mapping 29
9 Conclusion 29
Bibliography 30

Abstract

Blood banks and hospitals in many cities still maintain donor lists, stock registers and request records on paper or in separate spreadsheets. This makes it slow to find an eligible donor, to know how many units of each blood group are in stock, and to prove who approved a transfer of blood. This project presents **BloodLink**, a database-driven blood donation and inventory platform implemented entirely in Oracle SQL and PL/SQL and executed from SQL*Plus. The database stores donors, blood groups, blood banks, hospitals, donations, stock and hospital requests in eight tables designed up to Third Normal Form (3NF/BCNF) with primary key, foreign key, unique, check and not-null constraints.

Business rules are enforced inside the database: three triggers reject under-age donors, update the stock automatically when a donation is recorded, and write an audit trail whenever a request changes status. Views give ready-made reports such as stock per blood group and the list of donors who are eligible to donate today. Stored procedures and a function record donations and fulfil hospital requests as atomic transactions, and cursor-based procedures print low-stock and pending-request reports. The report explains the ACID properties, Oracle's row-level locking and read consistency, and the use of indexes, with a two-session test for concurrency control. All code is written to the Oracle 10g Release 2 feature set so that it also works on later releases.

Keywords: Blood donation, inventory management, Oracle, SQL*Plus, PL/SQL, normalization, triggers, cursors, transactions, concurrency control.

1 Introduction

Blood cannot be manufactured; it can only come from voluntary donors, and it has a limited shelf life. Hospitals therefore depend on blood banks having the right blood group in stock at the right time. The World Health Organization stresses that a reliable supply requires organised, voluntary donation and well-managed blood services [11]. When records are scattered, three problems appear: the same donor may be called again too soon after a donation, stock levels are not visible to the hospital that needs blood, and there is no reliable history of who issued which units.

A relational database management system (DBMS) solves these problems because it keeps one consistent copy of the data, enforces rules through constraints, allows many users to work at the same time, and guarantees that a transaction is either completed fully or not at all [1, 2]. This course end project applies those DBMS concepts to a realistic problem. **BloodLink** is a small but complete database application: its data model, SQL code, PL/SQL code and test results are all documented in this report.

1.1 Problem Statement

To design and implement a relational database in Oracle that records blood donors, donations, blood bank stock and hospital requests, keeps the stock correct when donations arrive and requests are served, and prevents invalid or unsafe operations such as under-age donors, donors who donated recently, and issuing more blood than is available.

1.2 Scope

BloodLink is a database back-end used through SQL*Plus; a graphical front-end is outside the scope of this course project. Donor names, phone numbers, hospitals and blood banks in the sample data are fictitious.

2 Literature Review

Relational model. Codd introduced the relational model, in which all data is stored in relations (tables) and accessed by value rather than by physical pointers [3]. He later defined the normal forms to remove redundancy and update anomalies [4]. BloodLink follows this approach: each fact is stored once and tables are linked by keys.

Conceptual design. Chen's entity–relationship (ER) model describes a system as entities, attributes and relationships and is the standard first step before creating tables [5]. Textbooks on database systems explain how to map an ER diagram to relations and how to check normal forms [1, 2].

Transactions and ACID. Haerder and Reuter named the Atomicity, Consistency, Isola-tion and Durability (ACID) properties and described how recovery mechanisms support them [6]. Gray and Reuter give the classical treatment of locking and transaction processing [7]. These ideas matter in a blood bank because two requests may compete for the last units of the same blood group.

Oracle features. Oracle documents its row-level locking, multi-version read consistency, B-tree indexes, triggers and PL/SQL language in its Database Concepts, SQL Reference and PL/SQL guides [8, 9, 10]. The implementation in this report uses only features described for Oracle Database 10g Release 2, so that it works unchanged on later releases.

Blood safety guidance. The World Health Organization and the Indian National AIDS Control Organisation (NACO) publish guidance for blood services, including donor selection and the minimum gap between whole-blood donations [11, 12]. BloodLink uses a configurable

rule of 90 days between donations; the exact interval should be confirmed against the guideline followed by the blood bank.

3 Objectives

To design an ER diagram and a relational schema for blood donation, stock and hospital requests, normalized to 3NF/BCNF.

To create the schema in Oracle with primary key, foreign key, unique, check and not-null constraints.

To load realistic sample data and answer typical questions using SELECT queries, joins, grouping and nested queries.

To create views for stock summaries, eligible donors and pending requests.

To use triggers for automatic stock update, age validation and audit logging.

To write PL/SQL procedures, a function and cursors for donation recording and request fulfilment.

To demonstrate transactions (COMMIT, SAVEPOINT, ROLLBACK), the ACID properties and concurrency control with row locks.

To use indexes to support common searches.

4 Methodology

4.1 Block Diagram

Figure 1 shows the architecture of BloodLink. Users (blood bank staff and hospital desk) work through SQL*Plus. SELECT statements go to views and tables, while operations that change data call PL/SQL procedures. Triggers fire automatically on the tables, and the Oracle transaction and lock manager guarantees correct concurrent behaviour.

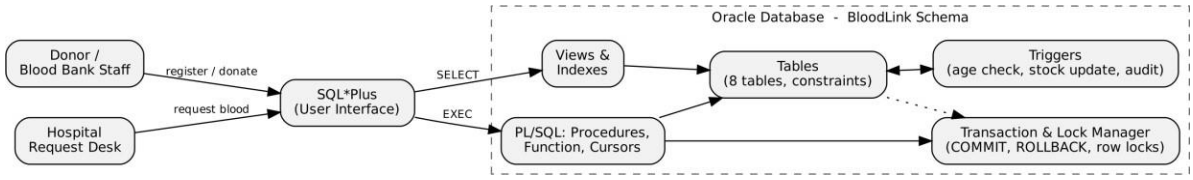


Figure 1: Block diagram of the BloodLink platform

4.2 ER Diagram

Figure 2 shows the entity–relationship diagram. The main entities are DONOR, BLOOD GROUP, BLOOD BANK, HOSPITAL, DONATION, INVENTORY and BLOOD REQUEST. AUDIT LOG is a system table written only by a trigger and therefore has no relationship line. Underlined attributes are primary keys; "1" and "N" show the cardinality of each relationship. The relationships are:

One blood group is the group of many donors (1:N); one donor makes many donations (1:N).

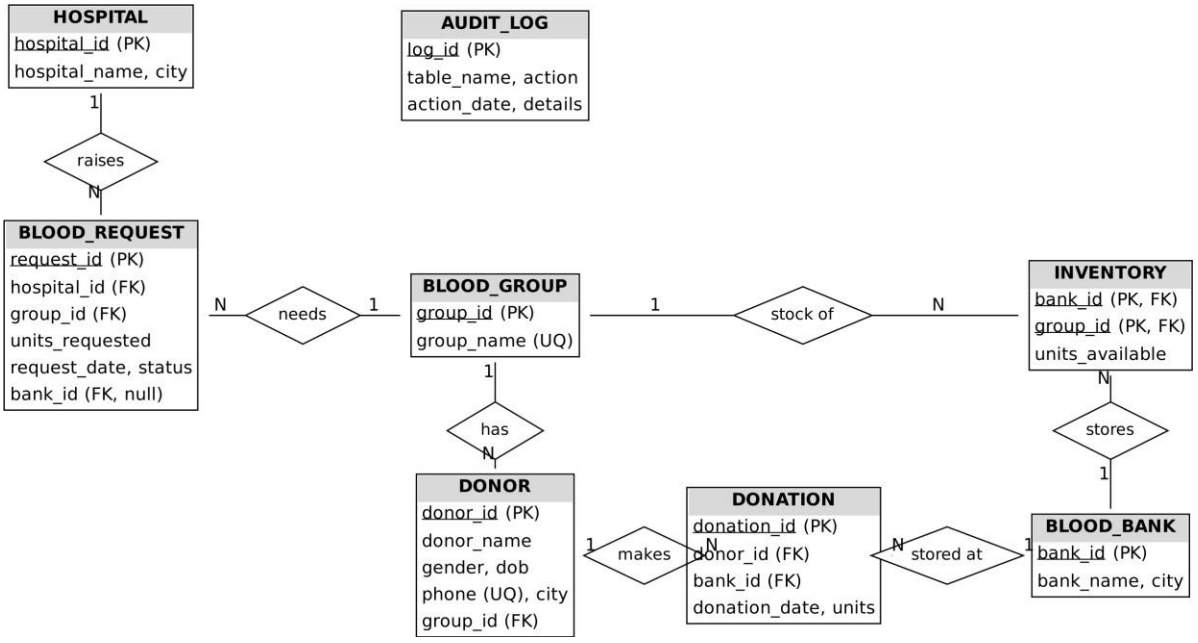


Figure 2: Entity–relationship diagram of BloodLink

One blood bank receives many donations (1:N) and keeps stock for each blood group. INVENTORY resolves this many-to-many relationship between BLOOD BANK and BLOOD GROUP; its primary key is the pair (bank_id, group_id).

One hospital raises many requests (1:N); one blood group is needed by many requests (1:N). A request records the serving bank only after it is fulfilled, so bank_id in BLOOD_REQUEST is optional.

4.3 Relational Schema

Relation	Attributes (PK underlined in text as [PK], FK as [FK])
BLOOD_GROUP	group_id [PK], group_name (unique)
BLOOD_BANK	bank_id [PK], bank_name, city
HOSPITAL	hospital_id [PK], hospital_name, city
DONOR	donor_id [PK], donor_name, gender, dob, phone (unique), city, group_id [FK → BLOOD_GROUP]
INVENTORY	(bank_id [FK], group_id [FK]) [PK], units_available
DONATION	donation_id [PK], donor_id [FK], bank_id [FK], donation_date, units
BLOOD_REQUEST	request_id [PK], hospital_id [FK], group_id [FK], units_requested, request_date, status, bank_id [FK, nullable]
AUDIT_LOG	log_id [PK], table_name, action, action_date, details

4.4 Normalization (3NF / BCNF)

Starting point (unnormalized). A single sheet DONOR_DONATION(donor_id, donor_name, phone, blood_group_name, bank_name, bank_city, donation_date, units) repeats the donor's name, phone and group in every donation row, and the bank name and city in every row for that bank. Changing a bank's city or a donor's phone would need many updates (update anomaly), and a donor with no donation could not be stored (insertion anomaly).

1NF. All attributes are atomic, with no repeating groups, and every row is identified by a key.

2NF. Attributes that depend on only part of the key are moved out. Donor details depend on donor_id alone, so they move to DONOR; bank details depend on bank_id alone, so they move to BLOOD_BANK. DONATION keeps only facts about the donation event.

3NF/BCNF. Transitive dependencies are removed: in DONOR, group id → group name, so the name of the blood group is stored once in BLOOD GROUP and DONOR keeps only the foreign key. Stock cannot be a column of a bank or a group alone, so it is stored in INVENTORY, keyed by (bank id, group_id). The last donation date of a donor is a derived value, so it is *not* stored in DONOR; it is computed with MAX(donation date) in the view V ELIGIBLE DONORS.

Functional dependencies and check.

BLOOD GROUP: group id → group name and group name → group id. Both determinants are candidate keys, so the relation is in BCNF.

DONOR: donor id → all other attributes; phone → donor id (phone is a candidate key through the UNIQUE constraint). Every determinant is a candidate key: BCNF.

INVENTORY: (bank id, group id) → units available. BCNF.

DONATION, BLOOD_REQUEST, BLOOD BANK, HOSPITAL, AUDIT LOG: the primary key is the only determinant: BCNF.

4.5 Database Concepts Used

Constraints: PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK (gender, phone length, units, status). All constraints are named so that error messages are readable.

Views: stored SELECT statements that hide joins and give reports.

Joins and nested queries: inner join, left outer join, subqueries with NOT IN, scalar and correlated subqueries, and an inline view with ROWNUM.

Triggers: row-level triggers on BEFORE INSERT and AFTER INSERT/UPDATE.

PL/SQL: functions, procedures, exceptions (RAISE APPLICATION ERROR) and cursors (explicit with parameter, and cursor FOR loop; the implicit cursor attribute SQL%ROWCOUNT is used inside a trigger).

Transactions and ACID: *Atomicity* – fulfill request updates stock and request together or not at all; *Consistency* – constraints and triggers keep the data valid (stock can never become negative); *Isolation* – Oracle row locks and read consistency; *Durability* – committed changes are written to the redo log [6, 8].

Concurrency control: Oracle readers never block writers and writers never block readers; two sessions that change the same row are serialized by a row-level lock. SELECT ... FOR UPDATE locks rows explicitly, and NOWAIT returns an error (ORA-00054) instead of waiting [8].

Indexing: Oracle automatically creates unique B-tree indexes for primary key and unique constraints; extra indexes are created on foreign keys and on frequently searched columns.

4.6 Tools Used

Tool	Purpose
Oracle Database [version, e.g. 10.2.0.1.0]	Database engine (code limited to Oracle 10g R2 features)
SQL*Plus	Command-line client to run scripts and queries
Graphviz	Drawing the block and ER diagrams
LaTeX / Overleaf	Preparing this report

4.7 How the Project is Organised

The SQL scripts are stored in the sql/ folder and are run in this order: 01_create_tables, 02_triggers, 03_insert_data, 03a_constraint_tests, 04_views, 05_indexes, 06_queries, 07_procedures, 08_transactions, 09_concurrency (two sessions), 10_run_tests, 11_dictionary_checks. The triggers are created before the data is loaded so that the donation rows in the sample data update the stock through the trigger.

5 Implementation

This section lists the SQL and PL/SQL source code. Each listing is a file in the sql/ folder of the project.

5.1 Tables, Constraints and Sequences

Eight tables are created with named constraints. Sequences supply new donation, request and audit-log numbers.

```
-- ============================================================
-- BloodLink : 01_create_tables.sql
-- Creates sequences and all tables (3NF) with constraints.
-- Run as: SQL> @01_create_tables.sql
-- Written for Oracle 10g Release 2 and later.
-- On the very first run the DROP statements show ORA-00942 /
-- ORA-02289 (object does not exist). This is harmless.
-- ============================================================
SET ECHO ON
DROP TABLE audit_log CASCADE CONSTRAINTS;
DROP TABLE blood_request CASCADE CONSTRAINTS;
DROP TABLE donation CASCADE CONSTRAINTS;
DROP TABLE inventory CASCADE CONSTRAINTS;
DROP TABLE donor CASCADE CONSTRAINTS;
DROP TABLE hospital CASCADE CONSTRAINTS;
DROP TABLE blood_bank CASCADE CONSTRAINTS;
DROP TABLE blood_group CASCADE CONSTRAINTS;
DROP SEQUENCE seq_donation;
DROP SEQUENCE seq_request;
DROP SEQUENCE seq_audit;

CREATE SEQUENCE seq_donation START WITH 11 INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE seq_request  START WITH 6  INCREMENT BY 1 NOCACHE;
CREATE SEQUENCE seq_audit    START WITH 1  INCREMENT BY 1 NOCACHE;

-- 1. Blood groups (lookup table)
CREATE TABLE blood_group (
    group_id    NUMBER(2)    CONSTRAINT pk_blood_group PRIMARY KEY,
    group_name  VARCHAR2(3)  NOT NULL
                CONSTRAINT uq_group_name UNIQUE
);

-- 2. Blood banks
CREATE TABLE blood_bank (
    bank_id     NUMBER(3)    CONSTRAINT pk_blood_bank PRIMARY KEY,
```

```
    bank_name   VARCHAR2(40)   NOT NULL,
    city        VARCHAR2(20)   NOT NULL
);

-- 3. Hospitals
CREATE TABLE hospital (
    hospital_id     NUMBER(3)      CONSTRAINT pk_hospital PRIMARY KEY,
    hospital_name   VARCHAR2(40)   NOT NULL,
    city            VARCHAR2(20)   NOT NULL
);

-- 4. Donors
CREATE TABLE donor (
    donor_id    NUMBER(4)      CONSTRAINT pk_donor PRIMARY KEY,
    donor_name  VARCHAR2(30)   NOT NULL,
    gender      CHAR(1)        NOT NULL
                CONSTRAINT ck_donor_gender CHECK (gender IN ('M','F','O')),
    dob         DATE           NOT NULL,
    phone       VARCHAR2(10)   NOT NULL
                CONSTRAINT uq_donor_phone UNIQUE
                CONSTRAINT ck_donor_phone CHECK (LENGTH(phone) = 10),
    city        VARCHAR2(20)   NOT NULL,
    group_id    NUMBER(2)      NOT NULL
                CONSTRAINT fk_donor_group REFERENCES blood_group(group_id)
);

-- 5. Inventory: units available per (blood bank, blood group)
CREATE TABLE inventory (
    bank_id          NUMBER(3)   CONSTRAINT fk_inv_bank  REFERENCES blood_bank(bank_id),
    group_id         NUMBER(2)   CONSTRAINT fk_inv_group REFERENCES blood_group(group_id),
    units_available  NUMBER(4)   DEFAULT 0 NOT NULL
                     CONSTRAINT ck_inv_units CHECK (units_available >= 0),
    CONSTRAINT pk_inventory PRIMARY KEY (bank_id, group_id)
);

-- 6. Donations
CREATE TABLE donation (
    donation_id     NUMBER(5)  CONSTRAINT pk_donation PRIMARY KEY,
    donor_id        NUMBER(4)  NOT NULL
                    CONSTRAINT fk_don_donor REFERENCES donor(donor_id),
    bank_id         NUMBER(3)  NOT NULL
                    CONSTRAINT fk_don_bank REFERENCES blood_bank(bank_id),
    donation_date   DATE       NOT NULL,
    units           NUMBER(2)  DEFAULT 1 NOT NULL
                    CONSTRAINT ck_don_units CHECK (units BETWEEN 1 AND 2)
);

-- 7. Blood requests raised by hospitals
CREATE TABLE blood_request (
    request_id      NUMBER(5)  CONSTRAINT pk_request PRIMARY KEY,
    hospital_id     NUMBER(3)  NOT NULL
                    CONSTRAINT fk_req_hospital REFERENCES hospital(hospital_id),
    group_id        NUMBER(2)  NOT NULL
                    CONSTRAINT fk_req_group REFERENCES blood_group(group_id),
    units_requested NUMBER(3)  NOT NULL
                    CONSTRAINT ck_req_units CHECK (units_requested > 0),
    request_date    DATE       NOT NULL,
    status          VARCHAR2(10) DEFAULT 'PENDING' NOT NULL
                    CONSTRAINT ck_req_status
                    CHECK (status IN ('PENDING','FULFILLED','REJECTED')),
    bank_id         NUMBER(3)
                    CONSTRAINT fk_req_bank REFERENCES blood_bank(bank_id)
);

-- 8. Audit log (filled automatically by a trigger)
CREATE TABLE audit_log (
    log_id      NUMBER(6)      CONSTRAINT pk_audit PRIMARY KEY,
    table_name  VARCHAR2(20)   NOT NULL,
    action      VARCHAR2(10)   NOT NULL,
    action_date DATE           DEFAULT SYSDATE NOT NULL,
    details     VARCHAR2(100)
);
```

Listing 1: 01_create_tables.sql

5.2 Triggers

trg_donor_age rejects donors younger than 18 years (216 months). trg_donation_inventory adds the donated units to the stock of the bank; it first tries an UPDATE and, if no stock row exists (SQL%ROWCOUNT = 0), inserts one. trg_request_audit writes one audit row whenever the status of a request changes.

```
-- ============================================================
-- BloodLink : 02_triggers.sql
-- Three row-level triggers. Run BEFORE 03_insert_data.sql.
-- ============================================================

-- Trigger 1: a donor must be at least 18 years old
CREATE OR REPLACE TRIGGER trg_donor_age
BEFORE INSERT ON donor
FOR EACH ROW
BEGIN
  IF MONTHS_BETWEEN(SYSDATE, :NEW.dob) < 216 THEN
    RAISE_APPLICATION_ERROR(-20001, 'Donor must be at least 18 years old');
  END IF;
END;
/
SHOW ERRORS

-- Trigger 2: every new donation increases the stock automatically
CREATE OR REPLACE TRIGGER trg_donation_inventory
AFTER INSERT ON donation
FOR EACH ROW
DECLARE
  v_group donor.group_id%TYPE;
BEGIN
  SELECT group_id INTO v_group
  FROM donor
  WHERE donor_id = :NEW.donor_id;

  UPDATE inventory
  SET units_available = units_available + :NEW.units
  WHERE bank_id = :NEW.bank_id AND group_id = v_group;

  IF SQL%ROWCOUNT = 0 THEN
    INSERT INTO inventory (bank_id, group_id, units_available)
    VALUES (:NEW.bank_id, v_group, :NEW.units);
  END IF;
END;
/
SHOW ERRORS

-- Trigger 3: audit every change of request status
CREATE OR REPLACE TRIGGER trg_request_audit
AFTER UPDATE OF status ON blood_request
FOR EACH ROW
BEGIN
  INSERT INTO audit_log (log_id, table_name, action, details)
  VALUES (seq_audit.NEXTVAL, 'BLOOD_REQUEST', 'UPDATE',
          'Request ' || :OLD.request_id || ': ' ||
          :OLD.status || ' -> ' || :NEW.status);
END;
/
SHOW ERRORS
```

Listing 2: 02_triggers.sql

5.3 Sample Data

```
-- ============================================================
-- BloodLink : 03_insert_data.sql
-- Sample data. Donation dates are relative to today so that the
-- eligibility rule (90 days) always gives the same result.
-- Trigger trg_donation_inventory adds donated units to stock.
-- ============================================================
INSERT INTO blood_group VALUES (1, 'A+');
INSERT INTO blood_group VALUES (2, 'A-');
INSERT INTO blood_group VALUES (3, 'B+');
INSERT INTO blood_group VALUES (4, 'B-');
INSERT INTO blood_group VALUES (5, 'AB+');
INSERT INTO blood_group VALUES (6, 'AB-');
INSERT INTO blood_group VALUES (7, 'O+');
INSERT INTO blood_group VALUES (8, 'O-');

INSERT INTO blood_bank VALUES (1, 'Hyderabad Central Blood Bank', 'Hyderabad');
INSERT INTO blood_bank VALUES (2, 'Secunderabad Community Blood Bank', 'Secunderabad');
INSERT INTO blood_bank VALUES (3, 'Warangal Regional Blood Bank', 'Warangal');

INSERT INTO hospital VALUES (1, 'Sunrise General Hospital', 'Hyderabad');
INSERT INTO hospital VALUES (2, 'Lifeline Medical Centre', 'Hyderabad');
INSERT INTO hospital VALUES (3, 'Green Valley Hospital', 'Warangal');
INSERT INTO hospital VALUES (4, 'Apex Trauma Centre', 'Secunderabad');

-- Donors (all names and phone numbers are fictitious)
INSERT INTO donor VALUES (101, 'Ravi Kumar',    'M', TO_DATE('14-05-1998','DD-MM-YYYY'), '9000000101', *
      Hyderabad',   1);
INSERT INTO donor VALUES (102, 'Anita Sharma', 'F', TO_DATE('02-11-1995','DD-MM-YYYY'), '9000000102', *
      Hyderabad',   7);
INSERT INTO donor VALUES (103, 'Suresh Reddy', 'M', TO_DATE('21-03-1990','DD-MM-YYYY'), '9000000103', *
      Warangal',    3);
INSERT INTO donor VALUES (104, 'Priya Nair',    'F', TO_DATE('30-07-2000','DD-MM-YYYY'), '9000000104', *
      Secunderabad', 8);
INSERT INTO donor VALUES (105, 'Imran Khan',    'M', TO_DATE('17-01-1992','DD-MM-YYYY'), '9000000105', *
      Hyderabad',   5);
INSERT INTO donor VALUES (106, 'Lakshmi Devi', 'F', TO_DATE('09-09-1988','DD-MM-YYYY'), '9000000106', *
      Warangal',    2);
INSERT INTO donor VALUES (107, 'Kiran Rao',     'M', TO_DATE('25-12-1999','DD-MM-YYYY'), '9000000107', *
      Hyderabad',   4);
INSERT INTO donor VALUES (108, 'Sneha Patel',  'F', TO_DATE('11-04-1997','DD-MM-YYYY'), '9000000108', *
      Secunderabad', 7);
INSERT INTO donor VALUES (109, 'Arjun Verma',  'M', TO_DATE('06-06-1993','DD-MM-YYYY'), '9000000109', *
      Hyderabad',   1);
INSERT INTO donor VALUES (110, 'Meena Iyer',    'F', TO_DATE('19-08-1991','DD-MM-YYYY'), '9000000110', *
      Warangal',    6);

-- Opening stock (before the donations below are recorded)
-- bank 1
INSERT INTO inventory VALUES (1, 1, 8);
INSERT INTO inventory VALUES (1, 2, 2);
INSERT INTO inventory VALUES (1, 3, 6);
INSERT INTO inventory VALUES (1, 4, 1);
INSERT INTO inventory VALUES (1, 5, 2);
INSERT INTO inventory VALUES (1, 6, 1);
INSERT INTO inventory VALUES (1, 7, 10);
INSERT INTO inventory VALUES (1, 8, 2);
-- bank 2
INSERT INTO inventory VALUES (2, 1, 5);
INSERT INTO inventory VALUES (2, 2, 1);
INSERT INTO inventory VALUES (2, 3, 4);
INSERT INTO inventory VALUES (2, 4, 0);
INSERT INTO inventory VALUES (2, 5, 1);
INSERT INTO inventory VALUES (2, 6, 0);
INSERT INTO inventory VALUES (2, 7, 6);
INSERT INTO inventory VALUES (2, 8, 1);
-- bank 3
INSERT INTO inventory VALUES (3, 1, 4);
INSERT INTO inventory VALUES (3, 2, 0);
INSERT INTO inventory VALUES (3, 3, 5);
INSERT INTO inventory VALUES (3, 4, 1);
INSERT INTO inventory VALUES (3, 5, 0);
INSERT INTO inventory VALUES (3, 6, 1);
```

```
63 INSERT INTO inventory VALUES (3, 7, 5);
64 INSERT INTO inventory VALUES (3, 8, 0);
65
66 -- Donations: (id, donor, bank, date = today - n days, units)
67 INSERT INTO donation VALUES (1, 101, 1, TRUNC(SYSDATE) - 200, 1);
68 INSERT INTO donation VALUES (2, 102, 1, TRUNC(SYSDATE) - 150, 1);
69 INSERT INTO donation VALUES (3, 103, 3, TRUNC(SYSDATE) - 100, 1);
70 INSERT INTO donation VALUES (4, 104, 2, TRUNC(SYSDATE) - 30, 1);
71 INSERT INTO donation VALUES (5, 105, 1, TRUNC(SYSDATE) - 95, 1);
72 INSERT INTO donation VALUES (6, 106, 3, TRUNC(SYSDATE) - 60, 1);
73 INSERT INTO donation VALUES (7, 108, 2, TRUNC(SYSDATE) - 180, 1);
74 INSERT INTO donation VALUES (8, 109, 1, TRUNC(SYSDATE) - 20, 1);
75 INSERT INTO donation VALUES (9, 102, 2, TRUNC(SYSDATE) - 250, 1);
76 INSERT INTO donation VALUES (10, 107, 1, TRUNC(SYSDATE) - 300, 1);
77
78 -- Requests from hospitals
79 INSERT INTO blood_request VALUES (1, 1, 7, 3, TRUNC(SYSDATE) - 5, 'PENDING', NULL);
80 INSERT INTO blood_request VALUES (2, 2, 1, 4, TRUNC(SYSDATE) - 3, 'PENDING', NULL);
81 INSERT INTO blood_request VALUES (3, 3, 4, 2, TRUNC(SYSDATE) - 2, 'PENDING', NULL);
82 INSERT INTO blood_request VALUES (4, 4, 6, 5, TRUNC(SYSDATE) - 1, 'PENDING', NULL);
83 INSERT INTO blood_request VALUES (5, 1, 8, 1, TRUNC(SYSDATE) - 10, 'FULFILLED', 1);
84
85 COMMIT;
```

Listing 3: 03_insert_data.sql

5.4 Constraint Tests

```
1 -- ============================================================
2 -- BloodLink : 03a_constraint_tests.sql
3 -- Every statement below is EXPECTED TO FAIL. Nothing is stored.
4 -- ============================================================
5 -- C1: CHECK constraint on phone length
6 INSERT INTO donor VALUES (111, 'Test User', 'M', TO_DATE('01-01-1990','DD-MM-YYYY'), '12345', 'Hyderabad
      ', 1);
7
8 -- C2: CHECK constraint on gender
9 INSERT INTO donor VALUES (111, 'Test User', 'X', TO_DATE('01-01-1990','DD-MM-YYYY'), '9000000111', '
      Hyderabad', 1);
10
11 -- C3: PRIMARY KEY violated (donor 101 already exists)
12 INSERT INTO donor VALUES (101, 'Test User', 'M', TO_DATE('01-01-1990','DD-MM-YYYY'), '9000000111', '
      Hyderabad', 1);
13
14 -- C4: FOREIGN KEY violated (blood group 99 does not exist)
15 INSERT INTO donor VALUES (111, 'Test User', 'M', TO_DATE('01-01-1990','DD-MM-YYYY'), '9000000111', '
      Hyderabad', 99);
16
17 -- C5: parent row cannot be deleted while child rows exist
18 DELETE FROM hospital WHERE hospital_id = 1;
19
20 -- C6: trigger TRG_DONOR_AGE rejects a minor
21 INSERT INTO donor VALUES (111, 'Test Minor', 'M', TO_DATE('01-01-2015','DD-MM-YYYY'), '9000000111', '
      Hyderabad', 1);
```

Listing 4: 03a_constraint_tests.sql

5.5 Views

```
1 -- ============================================================
2 -- BloodLink : 04_views.sql
3 -- ============================================================
4
5 -- View 1: stock per bank and blood group (joins 3 tables)
6 CREATE OR REPLACE VIEW v_inventory_summary AS
7 SELECT bb.bank_name, bg.group_name, i.units_available
8 FROM inventory i
9 JOIN blood_bank bb ON bb.bank_id = i.bank_id
```

```
JOIN blood_group bg ON bg.group_id = i.group_id;

-- View 2: total stock per blood group across all banks
CREATE OR REPLACE VIEW v_total_stock AS
SELECT bg.group_name, SUM(i.units_available) AS total_units
FROM inventory i
JOIN blood_group bg ON bg.group_id = i.group_id
GROUP BY bg.group_name;

-- View 3: donors who may donate now (no donation in last 90 days)
CREATE OR REPLACE VIEW v_eligible_donors AS
SELECT d.donor_id, d.donor_name, bg.group_name, d.city,
       TRUNC(SYSDATE) - MAX(dn.donation_date) AS days_since_last
FROM donor d
JOIN blood_group bg ON bg.group_id = d.group_id
LEFT JOIN donation dn ON dn.donor_id = d.donor_id
GROUP BY d.donor_id, d.donor_name, bg.group_name, d.city
HAVING MAX(dn.donation_date) IS NULL
    OR MAX(dn.donation_date) <= TRUNC(SYSDATE) - 90;

-- View 4: pending requests with hospital and blood group names
CREATE OR REPLACE VIEW v_pending_requests AS
SELECT r.request_id, h.hospital_name, bg.group_name, r.units_requested
FROM blood_request r
JOIN hospital h      ON h.hospital_id = r.hospital_id
JOIN blood_group bg ON bg.group_id    = r.group_id
WHERE r.status = 'PENDING';

---- ------------------- test the views -------------------
COLUMN bank_name      FORMAT A34
COLUMN group_name     FORMAT A10
COLUMN donor_name     FORMAT A15
COLUMN city           FORMAT A14
COLUMN hospital_name FORMAT A26

SELECT * FROM v_inventory_summary WHERE bank_name LIKE 'Hyderabad%' ORDER BY group_name;
SELECT * FROM v_total_stock ORDER BY group_name;
SELECT * FROM v_eligible_donors ORDER BY donor_id;
SELECT * FROM v_pending_requests ORDER BY request_id;
```

Listing 5: 04_views.sql

5.6 Indexes

```
-- ============================================================
-- BloodLink : 05_indexes.sql
-- Primary-key and UNIQUE constraints already create indexes.
-- These extra indexes speed up the most common searches.
-- ============================================================
CREATE INDEX idx_donor_group      ON donor(group_id);
CREATE INDEX idx_donor_city       ON donor(city);
CREATE INDEX idx_donation_donor   ON donation(donor_id);
CREATE INDEX idx_request_status   ON blood_request(status, group_id);
```

Listing 6: 05_indexes.sql

5.7 Queries (Joins, Aggregates, Nested Queries)

```
-- ============================================================
-- BloodLink : 06_queries.sql
-- SELECT queries: filtering, joins, grouping, nested queries.
-- ============================================================
SET LINESIZE 120
SET PAGESIZE 50
COLUMN donor_name     FORMAT A15
COLUMN group_name     FORMAT A10
COLUMN city           FORMAT A14
COLUMN bank_name      FORMAT A34
```

```
COLUMN hospital_name FORMAT A26

-- Q1: simple selection with WHERE and ORDER BY
SELECT donor_id, donor_name, city
FROM donor
WHERE city = 'Hyderabad'
ORDER BY donor_name;

-- Q2: INNER JOIN - donors with their blood group names
SELECT d.donor_id, d.donor_name, bg.group_name
FROM donor d
JOIN blood_group bg ON bg.group_id = d.group_id
ORDER BY d.donor_id;

-- Q3: GROUP BY - units collected by each blood bank
SELECT bb.bank_name, COUNT(*) AS donations, SUM(dn.units) AS units_collected
FROM donation dn
JOIN blood_bank bb ON bb.bank_id = dn.bank_id
GROUP BY bb.bank_name
ORDER BY bb.bank_name;

-- Q4: GROUP BY with HAVING - banks holding more than 20 units
SELECT bb.bank_name, SUM(i.units_available) AS total_units
FROM inventory i
JOIN blood_bank bb ON bb.bank_id = i.bank_id
GROUP BY bb.bank_name
HAVING SUM(i.units_available) > 20
ORDER BY total_units DESC;

-- Q5: nested query (NOT IN) - donors who have never donated
SELECT donor_id, donor_name
FROM donor
WHERE donor_id NOT IN (SELECT donor_id FROM donation);

-- Q6: correlated nested query - pending requests larger than
--     the total stock of that blood group in all banks
SELECT r.request_id, bg.group_name, r.units_requested,
       (SELECT SUM(i.units_available) FROM inventory i
        WHERE i.group_id = r.group_id) AS total_stock
FROM blood_request r
JOIN blood_group bg ON bg.group_id = r.group_id
WHERE r.status = 'PENDING'
  AND r.units_requested > (SELECT SUM(i.units_available)
                           FROM inventory i
                           WHERE i.group_id = r.group_id);

-- Q7: LEFT OUTER JOIN - number of requests raised by each hospital
SELECT h.hospital_name, COUNT(r.request_id) AS requests
FROM hospital h
LEFT JOIN blood_request r ON r.hospital_id = h.hospital_id
GROUP BY h.hospital_name
ORDER BY h.hospital_name;

-- Q8: inline view with ROWNUM - donor with the most donations
SELECT donor_name, donations
FROM (SELECT d.donor_name, COUNT(*) AS donations
      FROM donor d
      JOIN donation dn ON dn.donor_id = d.donor_id
      GROUP BY d.donor_name
      ORDER BY COUNT(*) DESC)
WHERE ROWNUM = 1;

-- Q9: find eligible donors for pending request 2 (blood group A+)
SELECT e.donor_id, e.donor_name, e.group_name
FROM v_eligible_donors e
WHERE e.group_name = (SELECT bg.group_name
                      FROM blood_request r
                      JOIN blood_group bg ON bg.group_id = r.group_id
                      WHERE r.request_id = 2);
```

Listing 7: 06_queries.sql

5.8 Functions, Procedures and Cursors

fn_is_eligible applies the 90-day rule. record_donation checks eligibility and inserts the donation. fulfill_request locks the request and stock rows with FOR UPDATE, checks the stock, reduces it, marks the request fulfilled and commits; on any error it rolls back. low_stock_report uses an explicit cursor with a parameter (OPEN, FETCH, %NOTFOUND, CLOSE), and pending_report uses a cursor FOR loop.

```
-- ============================================================
-- BloodLink : 07_procedures.sql
-- One function and four procedures (includes cursors,
-- exception handling and transaction control).
-- Run SET SERVEROUTPUT ON before calling them.
-- ============================================================

-- Function: can this donor donate today? (90-day rule)
CREATE OR REPLACE FUNCTION fn_is_eligible (p_donor_id IN NUMBER)
RETURN VARCHAR2
IS
   v_last   DATE;
BEGIN
   SELECT MAX(donation_date) INTO v_last
   FROM donation
   WHERE donor_id = p_donor_id;

   IF v_last IS NULL OR TRUNC(SYSDATE) - v_last >= 90 THEN
      RETURN 'YES';
   ELSE
      RETURN 'NO';
   END IF;
END;
/
SHOW ERRORS

-- Procedure 1: record a donation (trigger updates the stock)
CREATE OR REPLACE PROCEDURE record_donation (
   p_donor_id  IN  NUMBER,
   p_bank_id   IN  NUMBER,
   p_units     IN NUMBER DEFAULT 1)
IS
BEGIN
   IF fn_is_eligible(p_donor_id) = 'NO' THEN
      RAISE_APPLICATION_ERROR(-20002,
        'Donor is not eligible: last donation was within 90 days');
   END IF;

   INSERT INTO donation (donation_id, donor_id, bank_id, donation_date, units)
   VALUES (seq_donation.NEXTVAL, p_donor_id, p_bank_id, TRUNC(SYSDATE), p_units);

   COMMIT;
   DBMS_OUTPUT.PUT_LINE('Donation recorded for donor ' || p_donor_id);
END;
/
SHOW ERRORS

-- Procedure 2: fulfil a request from one blood bank (ACID transaction)
CREATE OR REPLACE PROCEDURE fulfill_request (
   p_request_id  IN  NUMBER,
   p_bank_id     IN  NUMBER)
IS
   v_group    blood_request.group_id%TYPE;
   v_units    blood_request.units_requested%TYPE;
   v_status   blood_request.status%TYPE;
   v_stock    inventory.units_available%TYPE;
BEGIN
   -- lock the request row so that no other session can change it
   SELECT group_id, units_requested, status
   INTO v_group, v_units, v_status
   FROM blood_request
   WHERE request_id = p_request_id
   FOR UPDATE;

```

```
    IF v_status <> 'PENDING' THEN
      ROLLBACK;
      RAISE_APPLICATION_ERROR(-20003, 'Request is not pending');
    END IF;

    -- lock the stock row of the chosen bank
    SELECT units_available INTO v_stock
    FROM inventory
    WHERE bank_id = p_bank_id AND group_id = v_group
    FOR UPDATE;

    IF v_stock < v_units THEN
      ROLLBACK;
      RAISE_APPLICATION_ERROR(-20004, 'Insufficient stock in selected bank');
    END IF;

    UPDATE inventory
    SET units_available = units_available - v_units
    WHERE bank_id = p_bank_id AND group_id = v_group;

    UPDATE blood_request
    SET status = 'FULFILLED', bank_id = p_bank_id
    WHERE request_id = p_request_id;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Request ' || p_request_id || ' fulfilled from bank '
                         || p_bank_id);
  EXCEPTION
    WHEN NO_DATA_FOUND THEN
      ROLLBACK;
      RAISE_APPLICATION_ERROR(-20005, 'Request or stock row not found');
  END;
  /
  SHOW ERRORS

  -- Procedure 3: explicit cursor with a parameter (OPEN-FETCH-CLOSE)
  CREATE OR REPLACE PROCEDURE low_stock_report (p_limit IN NUMBER)
  IS
    CURSOR c_low (c_limit NUMBER) IS
      SELECT bb.bank_name, bg.group_name, i.units_available
      FROM inventory i
      JOIN blood_bank bb ON bb.bank_id = i.bank_id
      JOIN blood_group bg ON bg.group_id = i.group_id
      WHERE i.units_available <= c_limit
      ORDER BY i.units_available, bb.bank_id, bg.group_id;
    v_bank   blood_bank.bank_name%TYPE;
    v_group  blood_group.group_name%TYPE;
    v_units  inventory.units_available%TYPE;
  BEGIN
    OPEN c_low(p_limit);
    LOOP
      FETCH c_low INTO v_bank, v_group, v_units;
      EXIT WHEN c_low%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE(v_bank || ' | ' || v_group || ' | ' || v_units);
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('Rows listed: ' || c_low%ROWCOUNT);
    CLOSE c_low;
  END;
  /
  SHOW ERRORS

  -- Procedure 4: cursor FOR loop - list pending requests
  CREATE OR REPLACE PROCEDURE pending_report
  IS
  BEGIN
    FOR r IN (SELECT request_id, hospital_name, group_name, units_requested
              FROM v_pending_requests
              ORDER BY request_id)
    LOOP
      DBMS_OUTPUT.PUT_LINE('Request ' || r.request_id || ': ' ||
        r.hospital_name || ' needs ' || r.units_requested ||
        ' unit(s) of ' || r.group_name);
    END LOOP;
```

```
END;
/
SHOW ERRORS
```

Listing 8: 07_procedures.sql

5.9 Transaction Control

```
-- ============================================================
-- BloodLink : 08_transactions.sql
-- COMMIT / SAVEPOINT / ROLLBACK demo on bank 1, group O- (8).
-- The script leaves the data unchanged at the end.
-- ============================================================
SET AUTOCOMMIT OFF

SELECT units_available FROM inventory WHERE bank_id = 1 AND group_id = 8;

UPDATE inventory SET units_available = units_available - 1
WHERE bank_id = 1 AND group_id = 8;

  SAVEPOINT    after_first_issue;

UPDATE inventory SET units_available = units_available - 1
WHERE bank_id = 1 AND group_id = 8;

SELECT units_available FROM inventory WHERE bank_id = 1 AND group_id = 8;

 ROLLBACK  TO  after_first_issue;

SELECT units_available FROM inventory WHERE bank_id = 1 AND group_id = 8;

-- Atomicity / consistency: this statement breaks CK_INV_UNITS (stock < 0)
UPDATE inventory SET units_available = units_available - 5
WHERE bank_id = 1 AND group_id = 8;

ROLLBACK;

SELECT units_available FROM inventory WHERE bank_id = 1 AND group_id = 8;
```

Listing 9: 08_transactions.sql

5.10 Concurrency Control (Two Sessions)

```
-- ============================================================
-- BloodLink : 09_concurrency.sql
-- Needs TWO SQL*Plus windows (Session 1 and Session 2), both
-- connected to the same user. Execute the steps in the order
-- of the step numbers. Row: bank 1, blood group O+ (7).
-- ============================================================

-- STEP 1 (Session 1): lock the stock row
SELECT units_available FROM inventory
WHERE bank_id = 1 AND group_id = 7 FOR UPDATE;

-- STEP 2 (Session 2): try to lock the same row without waiting
SELECT units_available FROM inventory
WHERE bank_id = 1 AND group_id = 7 FOR UPDATE NOWAIT;
-- expected: ORA-00054: resource busy and acquire with NOWAIT specified

-- STEP 3 (Session 2): an ordinary SELECT is never blocked
-- (Oracle read consistency): it returns the old committed value
SELECT units_available FROM inventory WHERE bank_id = 1 AND group_id = 7;

-- STEP 4 (Session 2): this UPDATE now WAITS (the window hangs)
UPDATE inventory SET units_available = units_available - 1
WHERE bank_id = 1 AND group_id = 7;

-- STEP 5 (Session 1): release the lock; Session 2 continues at once
```

```
ROLLBACK;

-- STEP 6 (Session 2): show the result, then undo the test
SELECT units_available FROM inventory WHERE bank_id = 1 AND group_id = 7;
ROLLBACK;
```

Listing 10: 09_concurrency.sql

5.11 Test Calls and Data Dictionary Checks

```
-- ============================================================
-- BloodLink : 10_run_tests.sql
-- Calls the functions/procedures and trigger tests.
-- Run AFTER 01 ... 07.
-- ============================================================
SET SERVEROUTPUT ON
SET LINESIZE 120
SET PAGESIZE 50
COLUMN donor_name FORMAT A15
COLUMN table_name FORMAT A15
COLUMN action     FORMAT A8
COLUMN details    FORMAT A35
COLUMN eligible   FORMAT A8

-- T1: function
SELECT donor_id, donor_name, fn_is_eligible(donor_id) AS eligible
FROM donor
ORDER BY donor_id;

-- T2: donation for an eligible donor (donor 110, bank 2)
EXEC record_donation(110, 2, 1);
SELECT units_available FROM inventory WHERE bank_id = 2 AND group_id = 6;

-- T3: donation for an ineligible donor (donor 104 gave 30 days ago)
EXEC record_donation(104, 1, 1);

-- T4: fulfil request 1 (3 units O+) from bank 1
EXEC fulfill_request(1, 1);
SELECT units_available FROM inventory WHERE bank_id = 1 AND group_id = 7;

-- T5: request 4 needs 5 units AB-, bank 1 has only 1
EXEC fulfill_request(4, 1);

-- T6: request 1 again (already fulfilled)
EXEC fulfill_request(1, 1);

-- T7: request 3 (2 units B-) from bank 1
EXEC fulfill_request(3, 1);

-- T8: audit trail written by trg_request_audit
SELECT log_id, table_name, action, details FROM audit_log ORDER BY log_id;

-- T9: cursor reports
EXEC low_stock_report(1);
EXEC pending_report;
```

Listing 11: 10_run_tests.sql

```
-- ============================================================
-- BloodLink : 11_dictionary_checks.sql
-- Checks tables, row counts, constraints and indexes using the
-- Oracle data dictionary. Run after 01 ... 05.
-- ============================================================
SET LINESIZE 120
SET PAGESIZE 50
COLUMN table_name      FORMAT A15
COLUMN constraint_name FORMAT A20
COLUMN index_name      FORMAT A20
COLUMN uniqueness      FORMAT A10
COLUMN constraint_type FORMAT A15
```

```
13
14 DESC donor
15
16 SELECT table_name FROM user_tables
17  WHERE table_name IN ('BLOOD_GROUP','BLOOD_BANK','HOSPITAL','DONOR',
18                       'INVENTORY','DONATION','BLOOD_REQUEST','AUDIT_LOG')
19 ORDER BY table_name;
20
21 SELECT 'BLOOD_GROUP' AS table_name, COUNT(*) AS row_count FROM blood_group
22  UNION ALL SELECT 'BLOOD_BANK',        COUNT(*) FROM blood_bank
23 UNION ALL SELECT 'HOSPITAL',          COUNT(*) FROM hospital
24  UNION ALL SELECT 'DONOR',           COUNT(*) FROM donor
25  UNION ALL SELECT 'INVENTORY',       COUNT(*) FROM inventory
26  UNION ALL SELECT 'DONATION',        COUNT(*) FROM donation
27  UNION ALL SELECT 'BLOOD_REQUEST', COUNT(*) FROM blood_request
28 UNION ALL SELECT 'AUDIT_LOG',        COUNT(*) FROM audit_log;
29
30  SELECT constraint_name, constraint_type
31  FROM user_constraints
32  WHERE table_name = 'DONOR' AND constraint_name NOT LIKE 'SYS%'
33  ORDER BY constraint_type, constraint_name;
34
35 SELECT index_name, table_name, uniqueness
36 FROM user_indexes
37  WHERE table_name IN ('BLOOD_GROUP','BLOOD_BANK','HOSPITAL','DONOR',
38                       'INVENTORY','DONATION','BLOOD_REQUEST','AUDIT_LOG')
39 ORDER BY table_name, index_name;
40
41 -- Execution plan (output depends on the optimizer; see report)
42 EXPLAIN PLAN FOR
43 SELECT donor_name FROM donor WHERE group_id = 7;
44 SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Listing 12: 11_dictionary_checks.sql

6 Execution and Results

Important note on the outputs. The Oracle database was *not* available in the environment where this report was prepared, so the code could not be executed there. The outputs below are **expected results**: they were worked out by hand from the sample data, and the SELECT queries were cross-checked with an equivalent SQLite database. Before submission, every test must be run in SQL*Plus and the screen output compared with (or replaced by) the real output. Small differences are normal: column spacing, the user name inside error messages (STUDENT is used here as an example), the line numbers in ORA-06512 messages, and the execution plan. Donation dates in the sample data are relative to the current date, so only integer "days since" values are displayed.

6.1 Test 1: Table Creation and Data Load

Aim: confirm that the eight tables exist and have the expected number of rows. **Com-mands:** @01 create tables, @02 triggers, @03 insert data, then @11 dictionary checks (the DESC, USER TABLES and row-count statements).

```
SQL> DESC donor
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------

 DONOR_ID                                  NOT NULL NUMBER(4)
 DONOR_NAME                                NOT NULL VARCHAR2(30)
 GENDER                                    NOT NULL CHAR(1)
 DOB                                       NOT NULL DATE
 PHONE                                     NOT NULL VARCHAR2(10)
 CITY                                      NOT NULL VARCHAR2(20)
 GROUP_ID                                  NOT NULL NUMBER(2)

SQL> SELECT table_name FROM user_tables
```

```
  2  WHERE table_name IN ('BLOOD_GROUP','BLOOD_BANK','HOSPITAL','DONOR',
  3                       'INVENTORY','DONATION','BLOOD_REQUEST','AUDIT_LOG')
  4  ORDER BY table_name;

TABLE_NAME
--------------------------
AUDIT_LOG
BLOOD_BANK
BLOOD_GROUP
BLOOD_REQUEST
DONATION
DONOR
HOSPITAL
INVENTORY

8 rows selected.


SQL> SELECT 'BLOOD_GROUP' AS table_name, COUNT(*) AS row_count FROM blood_group
  2  UNION ALL SELECT 'BLOOD_BANK',     COUNT(*) FROM blood_bank
  3  UNION ALL SELECT 'HOSPITAL',       COUNT(*) FROM hospital
  4  UNION ALL SELECT 'DONOR',          COUNT(*) FROM donor
  5  UNION ALL SELECT 'INVENTORY',      COUNT(*) FROM inventory
  6  UNION ALL SELECT 'DONATION',       COUNT(*) FROM donation
  7  UNION ALL SELECT 'BLOOD_REQUEST', COUNT(*) FROM blood_request
  8  UNION ALL SELECT 'AUDIT_LOG',      COUNT(*) FROM audit_log;

TABLE_NAME         ROW_COUNT
-------------------- ----------
BLOOD_GROUP                 8
BLOOD_BANK                  3
HOSPITAL                    4
DONOR                      10
INVENTORY                  24
DONATION                   10
BLOOD_REQUEST               5
AUDIT_LOG                   0

8 rows selected.
```

6.2 Test 2: Constraints and Trigger Validation

Aim: show that invalid data is rejected. Constraints of the DONOR table are listed first, then six statements that must fail (@03a constraint_tests).

```
SQL> SELECT constraint_name, constraint_type
  2  FROM user_constraints
  3  WHERE table_name = 'DONOR' AND constraint_name NOT LIKE 'SYS%'
  4  ORDER BY constraint_type, constraint_name;

CONSTRAINT_NAME                  CONSTRAINT_TYPE
-------------------------------- ---------------
CK_DONOR_GENDER                  C
CK_DONOR_PHONE                   C
PK_DONOR                         P
FK_DONOR_GROUP                   R
UQ_DONOR_PHONE                   U


SQL> INSERT INTO donor VALUES (111, 'Test User', 'M', TO_DATE('01-01-1990','DD-MM-YYYY'), '12345', '
     Hyderabad', 1);
*
ERROR at line 1:
ORA-02290: check constraint (STUDENT.CK_DONOR_PHONE) violated

SQL> INSERT INTO donor VALUES (111, 'Test User', 'X', TO_DATE('01-01-1990','DD-MM-YYYY'), '9000000111', '
     Hyderabad', 1);
*
ERROR at line 1:
ORA-02290: check constraint (STUDENT.CK_DONOR_GENDER) violated
```

```
SQL> INSERT INTO donor VALUES (101, 'Test User', 'M', TO_DATE('01-01-1990','DD-MM-YYYY'), '9000000111', '
     Hyderabad', 1);
*
ERROR at line 1:
ORA-00001: unique constraint (STUDENT.PK_DONOR) violated

SQL> INSERT INTO donor VALUES (111, 'Test User', 'M', TO_DATE('01-01-1990','DD-MM-YYYY'), '9000000111', '
     Hyderabad', 99);
*
ERROR at line 1:
ORA-02291: integrity constraint (STUDENT.FK_DONOR_GROUP) violated - parent key not found

SQL> DELETE FROM hospital WHERE hospital_id = 1;
*
ERROR at line 1:
ORA-02292: integrity constraint (STUDENT.FK_REQ_HOSPITAL) violated - child record found

SQL> INSERT INTO donor VALUES (111, 'Test Minor', 'M', TO_DATE('01-01-2015','DD-MM-YYYY'), '9000000111', '
     Hyderabad', 1);
*
ERROR at line 1:
ORA-20001: Donor must be at least 18 years old
ORA-06512: at "STUDENT.TRG_DONOR_AGE", line 3
ORA-04088: error during execution of trigger 'STUDENT.TRG_DONOR_AGE'
```

6.3 Test 3: SELECT Queries, Joins and Nested Queries

Aim: answer typical questions: donors by city (Q1), join with blood group (Q2), units collected per bank (Q3), banks with more than 20 units using HAVING (Q4), donors who never donated using NOT IN (Q5), requests larger than the total stock using a correlated subquery (Q6), requests per hospital using a LEFT JOIN (Q7), top donor using an inline view and ROWNUM (Q8), and eligible donors for request 2 (Q9). **Command:** @06_queries.

```
SQL> SELECT donor_id, donor_name, city
  2  FROM donor
  3  WHERE city = 'Hyderabad'
  4  ORDER BY donor_name;

  DONOR_ID DONOR_NAME       CITY
---------- --------------- --------------------
       102 Anita Sharma    Hyderabad
       109 Arjun Verma     Hyderabad
       105 Imran Khan      Hyderabad
       107 Kiran Rao       Hyderabad
       101 Ravi Kumar      Hyderabad


SQL> SELECT d.donor_id, d.donor_name, bg.group_name
  2  FROM donor d
  3  JOIN blood_group bg ON bg.group_id = d.group_id
  4  ORDER BY d.donor_id;

  DONOR_ID DONOR_NAME          GROUP_NAME
---------- -------------------- ---------------
       101 Ravi Kumar           A+
       102 Anita Sharma         O+
       103 Suresh Reddy         B+
       104 Priya Nair           O-
       105 Imran Khan           AB+
       106 Lakshmi Devi         A-
       107 Kiran Rao            B-
       108 Sneha Patel          O+
       109 Arjun Verma          A+
       110 Meena Iyer           AB-

10 rows selected.


SQL> SELECT bb.bank_name, COUNT(*) AS donations, SUM(dn.units) AS units_collected
  2  FROM donation dn
```

```
  3  JOIN blood_bank bb ON bb.bank_id = dn.bank_id
  4  GROUP BY bb.bank_name
  5  ORDER BY bb.bank_name;

BANK_NAME                                                  DONATIONS UNITS_COLLECTED
---------------------------------------------------------- ---------- ---------------
Hyderabad Central Blood Bank                                        5               5
Secunderabad Community Blood Bank                                   3               3
Warangal Regional Blood Bank                                        2               2


SQL> SELECT bb.bank_name, SUM(i.units_available) AS total_units
  2  FROM inventory i
  3  JOIN blood_bank bb ON bb.bank_id = i.bank_id
  4  GROUP BY bb.bank_name
  5  HAVING SUM(i.units_available) > 20
  6  ORDER BY total_units DESC;

BANK_NAME                                                  TOTAL_UNITS
---------------------------------------------------------- -----------
Hyderabad Central Blood Bank                                        37
Secunderabad Community Blood Bank                                   21


SQL> SELECT donor_id, donor_name
  2  FROM donor
  3  WHERE donor_id NOT IN (SELECT donor_id FROM donation);

  DONOR_ID DONOR_NAME
---------- --------------------------
       110 Meena Iyer


SQL> SELECT r.request_id, bg.group_name, r.units_requested,
  2         (SELECT SUM(i.units_available) FROM inventory i
  3          WHERE i.group_id = r.group_id) AS total_stock
  4  FROM blood_request r
  5  JOIN blood_group bg ON bg.group_id = r.group_id
  6  WHERE r.status = 'PENDING'
  7    AND r.units_requested > (SELECT SUM(i.units_available)
  8                             FROM inventory i
  9                             WHERE i.group_id = r.group_id);

REQUEST_ID GROUP_NAME UNITS_REQUESTED TOTAL_STOCK
---------- ---------- --------------- -----------
         4 AB-                      5           2


SQL> SELECT h.hospital_name, COUNT(r.request_id) AS requests
  2  FROM hospital h
  3  LEFT JOIN blood_request r ON r.hospital_id = h.hospital_id
  4  GROUP BY h.hospital_name
  5  ORDER BY h.hospital_name;

HOSPITAL_NAME                  REQUESTS
------------------------------ ----------
Apex Trauma Centre                      1
Green Valley Hospital                   1
Lifeline Medical Centre                 1
Sunrise General Hospital                2


SQL> SELECT donor_name, donations
  2  FROM (SELECT d.donor_name, COUNT(*) AS donations
  3        FROM donor d
  4        JOIN donation dn ON dn.donor_id = d.donor_id
  5        GROUP BY d.donor_name
  6        ORDER BY COUNT(*) DESC)
  7  WHERE ROWNUM = 1;

DONOR_NAME          DONATIONS
--------------------------- ----------
Anita Sharma                         2
```

```
SQL> SELECT e.donor_id, e.donor_name, e.group_name
  2  FROM v_eligible_donors e
  3  WHERE e.group_name = (SELECT bg.group_name
  4                        FROM blood_request r
  5                        JOIN blood_group bg ON bg.group_id = r.group_id
  6                        WHERE r.request_id = 2);

  DONOR_ID DONOR_NAME          GROUP_NAME
---------- ------------------------- ----------
       101 Ravi Kumar           A+
```

6.4 Test 4: Views

Command: @04 views (the file creates the views and then queries them). The column DAYS_SINCE_LAST is empty for donor 110 because that donor has never donated.

```
SQL> SELECT * FROM v_inventory_summary WHERE bank_name LIKE 'Hyderabad%' ORDER BY group_name;

BANK_NAME                                GROUP_NAME UNITS_AVAILABLE
---------------------------------------- ---------- ---------------
Hyderabad Central Blood Bank             A+                      10
Hyderabad Central Blood Bank             A-                       2
Hyderabad Central Blood Bank             AB+                      3
Hyderabad Central Blood Bank             AB-                      1
Hyderabad Central Blood Bank             B+                       6
Hyderabad Central Blood Bank             B-                       2
Hyderabad Central Blood Bank             O+                      11
Hyderabad Central Blood Bank             O-                       2

8 rows selected.


SQL> SELECT * FROM v_total_stock ORDER BY group_name;

GROUP_NAME TOTAL_UNITS
---------- -----------
A+                  19
A-                   4
AB+                  4
AB-                  2
B+                  16
B-                   3
O+                  24
O-                   4

8 rows selected.


SQL> SELECT * FROM v_eligible_donors ORDER BY donor_id;

  DONOR_ID DONOR_NAME          GROUP_NAME CITY            DAYS_SINCE_LAST
---------- ------------------------- ---------- ------------------ ---------------
       101 Ravi Kumar           A+         Hyderabad              200
       102 Anita Sharma         O+         Hyderabad              150
       103 Suresh Reddy         B+         Warangal               100
       105 Imran Khan           AB+        Hyderabad               95
       107 Kiran Rao            B-         Hyderabad              300
       108 Sneha Patel          O+         Secunderabad           180
       110 Meena Iyer           AB-        Warangal

7 rows selected.


SQL> SELECT * FROM v_pending_requests ORDER BY request_id;

REQUEST_ID HOSPITAL_NAME                            GROUP_NAME UNITS_REQUESTED
---------- ---------------------------------------- ---------- ---------------
         1 Sunrise General Hospital                 O+                      3
         2 Lifeline Medical Centre                  A+                      4
```

```
         3 Green Valley Hospital        B-                    2
         4 Apex Trauma Centre           AB-                   5
```

6.5 Test 5: Indexes

Command: @05_indexes, then the USER_INDEXES query from 11_dictionary_checks. The indexes with SYS_ names are avoided because every constraint was named.

```
SQL> SELECT index_name, table_name, uniqueness
  2  FROM user_indexes
  3  WHERE table_name IN ('BLOOD_GROUP','BLOOD_BANK','HOSPITAL','DONOR',
  4                       'INVENTORY','DONATION','BLOOD_REQUEST','AUDIT_LOG')
  5  ORDER BY table_name, index_name;

INDEX_NAME                     TABLE_NAME          UNIQUENESS
------------------------------ ------------------- ----------
PK_AUDIT                       AUDIT_LOG           UNIQUE
PK_BLOOD_BANK                  BLOOD_BANK          UNIQUE
PK_BLOOD_GROUP                 BLOOD_GROUP         UNIQUE
UQ_GROUP_NAME                  BLOOD_GROUP         UNIQUE
IDX_REQUEST_STATUS             BLOOD_REQUEST       NONUNIQUE
PK_REQUEST                     BLOOD_REQUEST       UNIQUE
IDX_DONATION_DONOR             DONATION            NONUNIQUE
PK_DONATION                    DONATION            UNIQUE
IDX_DONOR_CITY                 DONOR               NONUNIQUE
IDX_DONOR_GROUP                DONOR               NONUNIQUE
PK_DONOR                       DONOR               UNIQUE
UQ_DONOR_PHONE                 DONOR               UNIQUE
PK_HOSPITAL                    HOSPITAL            UNIQUE
PK_INVENTORY                   INVENTORY           UNIQUE

14 rows selected.
```

The file 11_dictionary_checks.sql also contains EXPLAIN PLAN FOR SELECT donor_name FROM donor WHERE group_id = 7; followed by SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);. The plan is chosen by the optimizer: with only ten rows Oracle may prefer a full table scan (TABLE ACCESS FULL) instead of INDEX RANGE SCAN on IDX_DONOR_GROUP. **Run this test yourself and paste your real plan or a screenshot here**; no plan output is shown in this report because it cannot be predicted reliably.

6.6 Test 6: Function, Procedures, Cursors and Triggers

Aim: check the eligibility function, donation recording (trigger updates the stock), request fulfilment with the three outcomes (success, insufficient stock, already fulfilled), the audit trigger and the two cursor reports. **Commands:** @07 procedures, then @10 run_tests. The results depend on running the tests in this order.

```
SQL> SELECT donor_id, donor_name, fn_is_eligible(donor_id) AS eligible
  2  FROM donor
  3  ORDER BY donor_id;

  DONOR_ID DONOR_NAME       ELIGIBLE
---------- ---------------- ----------
       101 Ravi Kumar       YES
       102 Anita Sharma     YES
       103 Suresh Reddy     YES
       104 Priya Nair       NO
       105 Imran Khan       YES
       106 Lakshmi Devi     NO
       107 Kiran Rao        YES
       108 Sneha Patel      YES
       109 Arjun Verma      NO
       110 Meena Iyer       YES

10 rows selected.
```

```
SQL> EXEC record_donation(110, 2, 1);
Donation recorded for donor 110

PL/SQL procedure successfully completed.

SQL> SELECT units_available FROM inventory WHERE bank_id = 2 AND group_id = 6;

UNITS_AVAILABLE
---------------------------
                  1


SQL> EXEC record_donation(104, 1, 1);
BEGIN record_donation(104, 1, 1); END;
*
ERROR at line 1:
ORA-20002: Donor is not eligible: last donation was within 90 days
ORA-06512: at "STUDENT.RECORD_DONATION", line 8
ORA-06512: at line 1

SQL> EXEC fulfill_request(1, 1);
Request 1 fulfilled from bank 1

PL/SQL procedure successfully completed.

SQL> SELECT units_available FROM inventory WHERE bank_id = 1 AND group_id = 7;

UNITS_AVAILABLE
---------------------------
                  8


SQL> EXEC fulfill_request(4, 1);
BEGIN fulfill_request(4, 1); END;
*
ERROR at line 1:
ORA-20004: Insufficient stock in selected bank
ORA-06512: at "STUDENT.FULFILL_REQUEST", line 30
ORA-06512: at line 1

SQL> EXEC fulfill_request(1, 1);
BEGIN fulfill_request(1, 1); END;
*
ERROR at line 1:
ORA-20003: Request is not pending
ORA-06512: at "STUDENT.FULFILL_REQUEST", line 19
ORA-06512: at line 1

SQL> EXEC fulfill_request(3, 1);
Request 3 fulfilled from bank 1

PL/SQL procedure successfully completed.

SQL> SELECT log_id, table_name, action, details FROM audit_log ORDER BY log_id;

    LOG_ID TABLE_NAME      ACTION    DETAILS
---------- --------------- --------- ----------------------------------------
         1 BLOOD_REQUEST   UPDATE    Request 1: PENDING -> FULFILLED
         2 BLOOD_REQUEST   UPDATE    Request 3: PENDING -> FULFILLED


SQL> EXEC low_stock_report(1);
Hyderabad Central Blood Bank | B- | 0
Secunderabad Community Blood Bank | B- | 0
Warangal Regional Blood Bank | AB+ | 0
Warangal Regional Blood Bank | O- | 0
Hyderabad Central Blood Bank | AB- | 1
Secunderabad Community Blood Bank | A- | 1
Secunderabad Community Blood Bank | AB+ | 1
Secunderabad Community Blood Bank | AB- | 1
Warangal Regional Blood Bank | A- | 1
Warangal Regional Blood Bank | B- | 1
```

```
Warangal Regional Blood Bank | AB- | 1
Rows listed: 11

PL/SQL procedure successfully completed.

SQL> EXEC pending_report;
Request 2: Lifeline Medical Centre needs 4 unit(s) of A+
Request 4: Apex Trauma Centre needs 5 unit(s) of AB-

PL/SQL procedure successfully completed.
```

6.7 Test 7: Transactions (COMMIT, SAVEPOINT, ROLLBACK)

Command: @08_transactions. Stock of O− in bank 1 goes 2 → 1 → 0, returns to 1 at the savepoint, and returns to 2 after the final ROLLBACK. The failed UPDATE shows consistency: a statement that would make the stock negative is rejected by the check constraint.

```
SQL> SELECT units_available FROM inventory WHERE bank_id = 1 AND group_id = 8;

UNITS_AVAILABLE
---------------------------
                  2


SQL> UPDATE inventory SET units_available = units_available - 1
  2  WHERE bank_id = 1 AND group_id = 8;

1 row updated.

SQL> SAVEPOINT after_first_issue;

Savepoint created.

SQL> UPDATE inventory SET units_available = units_available - 1
  2  WHERE bank_id = 1 AND group_id = 8;

1 row updated.

SQL> SELECT units_available FROM inventory WHERE bank_id = 1 AND group_id = 8;

UNITS_AVAILABLE
---------------------------
                  0


SQL> ROLLBACK TO after_first_issue;

Rollback complete.

SQL> SELECT units_available FROM inventory WHERE bank_id = 1 AND group_id = 8;

UNITS_AVAILABLE
---------------------------
                  1


SQL> UPDATE inventory SET units_available = units_available - 5
  2  WHERE bank_id = 1 AND group_id = 8;
UPDATE inventory SET units_available = units_available - 5
*
ERROR at line 1:
ORA-02290: check constraint (STUDENT.CK_INV_UNITS) violated

SQL> ROLLBACK;

Rollback complete.

SQL> SELECT units_available FROM inventory WHERE bank_id = 1 AND group_id = 8;

UNITS_AVAILABLE
---------------------------
```

```
                2
```

6.8 Test 8: Concurrency Control (Two SQL*Plus Sessions)

Commands: the steps in 09_concurrency.sql, run in two windows. Session 1 locks the row; Session 2 cannot lock it with NOWAIT, can still read the old committed value, and its UPDATE waits until Session 1 ends its transaction. Run @09_concurrency steps manually after @08 transactions and before @10 run_tests so that the stock value is 11; if you run it later, the value will be 8.

```
-- ===== STEP 1 : SESSION 1 =====
SQL> SELECT units_available FROM inventory
  2  WHERE bank_id = 1 AND group_id = 7 FOR UPDATE;

UNITS_AVAILABLE
---------------------------
                11


-- ===== STEP 2 : SESSION 2 =====
SQL> SELECT units_available FROM inventory
  2  WHERE bank_id = 1 AND group_id = 7 FOR UPDATE NOWAIT;
SELECT  units_available  FROM  inventory
*
ERROR at line 1:
ORA-00054: resource busy and acquire with NOWAIT specified

-- ===== STEP 3 : SESSION 2 =====
SQL> SELECT units_available FROM inventory WHERE bank_id = 1 AND group_id = 7;

UNITS_AVAILABLE
---------------------------
                11


-- ===== STEP 4 : SESSION 2 (window hangs until Session 1 ends its transaction) =====
SQL> UPDATE inventory SET units_available = units_available - 1
  2  WHERE bank_id = 1 AND group_id = 7;

-- ===== STEP 5 : SESSION 1 =====
SQL> ROLLBACK;

Rollback  complete.

-- ===== STEP 4 continues : SESSION 2 =====
1 row updated.

-- ===== STEP 6 : SESSION 2 =====
SQL> SELECT units_available FROM inventory WHERE bank_id = 1 AND group_id = 7;

UNITS_AVAILABLE
---------------------------
                10


SQL> ROLLBACK;

Rollback  complete.
```

7 Results and Discussion

7.1 Summary of Tests

No.	Feature tested	Expected behaviour	Status
1	Tables, data load	8 tables; 8/3/4/10/24/10/5/0 rows	To verify
2	Constraints, age trigger	Six invalid statements rejected with ORA-02290, 00001, 02291, 02292, 20001	To verify
3	Queries Q1–Q9	Joins, grouping and subqueries return the listed rows	To verify
4	Views	Four views return stock, eligible donors, pending requests	To verify
5	Indexes	14 indexes, 4 created explicitly	To verify
6	PL/SQL and triggers	Stock updated by trigger; fulfilment atomic; audit rows written	To verify
7	Transactions	ROLLBACK restores stock; negative stock rejected	To verify
8	Concurrency	NOWAIT gives ORA-00054; UPDATE waits	To verify

The “To verify” status means that the result was derived by calculation and must be confirmed by running the scripts in Oracle.

7.2 Discussion

Data integrity is enforced by the database. Invalid phone numbers, genders, unknown blood groups, duplicate donors and under-age donors are rejected no matter which program or user sends the statement.

Automatic stock. Because the stock is updated by a trigger inside the same transaction as the donation, the stock cannot be forgotten or entered twice by mistake.

Safe issue of blood. `fulfill` request locks the request and the stock row, checks the available units and changes both tables together. When request 4 asks for 5 units of AB− but only 1 unit is in bank 1, the request is refused and nothing changes.

Derived data is not stored. The last donation date is computed in a view, so it can never become inconsistent with the donation table.

Indexes. The foreign-key and status indexes help the joins and the searches for pending requests when the tables grow. On the tiny sample tables the optimizer may ignore them, which is normal.

7.3 Limitations and Future Work

The project has no graphical interface and no user login; Oracle users and privileges (GRANT) could be added for staff, hospitals and administrators. Blood components, expiry dates of units, donor screening results and automatic selection of the nearest bank are not modelled.

8 Mapping POs and SDGs

8.1 Program Outcomes (POs) Mapping

Table 1: Mapping of Program Outcomes (POs) to Project Relevance

PO No.	Program Outcome	Relevance
PO1	Engineering Knowledge	Applied relational model, normalization, SQL, PL/SQL and transaction concepts.
PO2	Problem Analysis	Identified problems of manual blood donor and stock records and converted them to ER model and FDs.
PO3	Design/Development of Solutions	Designed the schema, triggers and procedures for a working blood donation and inventory database.
PO5	Modern Tool Usage	Used Oracle, SQL*Plus, Graphviz and Overleaf.
PO6	Engineer and Society	Supports faster, more reliable blood availability for hospitals and patients.
PO9	Individual and Team Work	Work divided among three team members for design, coding and testing.
PO10	Communication	Prepared a structured technical report and test documentation.

8.2 Sustainable Development Goals (SDGs) Mapping

Table 2: Mapping of Sustainable Development Goals (SDGs) to Project Relevance

SDG No.	Goal	Relevance
SDG 3	Good Health and Well-being	Helps hospitals find the right blood group quickly and keeps donors safe through eligibil-ity rules.
SDG 9	Industry, Innovation, and Infrastructure	Provides a reliable digital record system for health-care infrastructure.
SDG 16	Peace, Justice and Strong Institutions	Audit log gives transparent, accountable records of blood issue.

9 Conclusion

BloodLink demonstrates how the main DBMS concepts work together in one application: an ER design normalized to BCNF, a constrained Oracle schema, views and indexes, joins and nested queries, triggers, PL/SQL procedures with cursors, and transactions that follow the ACID properties under concurrent use. The database itself prevents invalid donors, unsafe donations and over-issue of blood. Future work can add user roles, unit expiry tracking and a web front-end on top of the same database.

Bibliography

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed., McGraw-Hill Education, 2019.

[2] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed., Pearson, 2016.

[3] E. F. Codd, ”A Relational Model of Data for Large Shared Data Banks,” *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[4] E. F. Codd, ”Further Normalization of the Data Base Relational Model,” in *Data Base Systems*, Courant Computer Science Symposia Series, vol. 6, Prentice-Hall, 1972, pp. 33–64.

[5] P. P.-S. Chen, ”The Entity-Relationship Model—Toward a Unified View of Data,” *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9–36, 1976.

[6] T. Haerder and A. Reuter, ”Principles of Transaction-Oriented Database Recovery,” *ACM Computing Surveys*, vol. 15, no. 4, pp. 287–317, 1983.

[7] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*, Morgan Kaufmann, 1993.

[8] Oracle Corporation, *Oracle Database Concepts, 10g Release 2 (10.2)*, Part No. B14220.

[9] Oracle Corporation, *Oracle Database SQL Reference, 10g Release 2 (10.2)*, Part No. B14200.

[10] Oracle Corporation, *Oracle Database PL/SQL User’s Guide and Reference, 10g Release 2 (10.2)*, Part No. B14261.

[11] World Health Organization, ”Blood safety and availability,” Fact sheet. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/blood-safety-and-availability

[12] National AIDS Control Organisation (NACO), Ministry of Health and Family Welfare, Government of India, *Standards for Blood Banks and Blood Transfusion Services*, 2007.